

Star Health

Star Health AI Tutor

A Proof of Concept (POC) for voice-to-voice AI tutoring

v1.0 — POC · Sept 2026

Submitted By



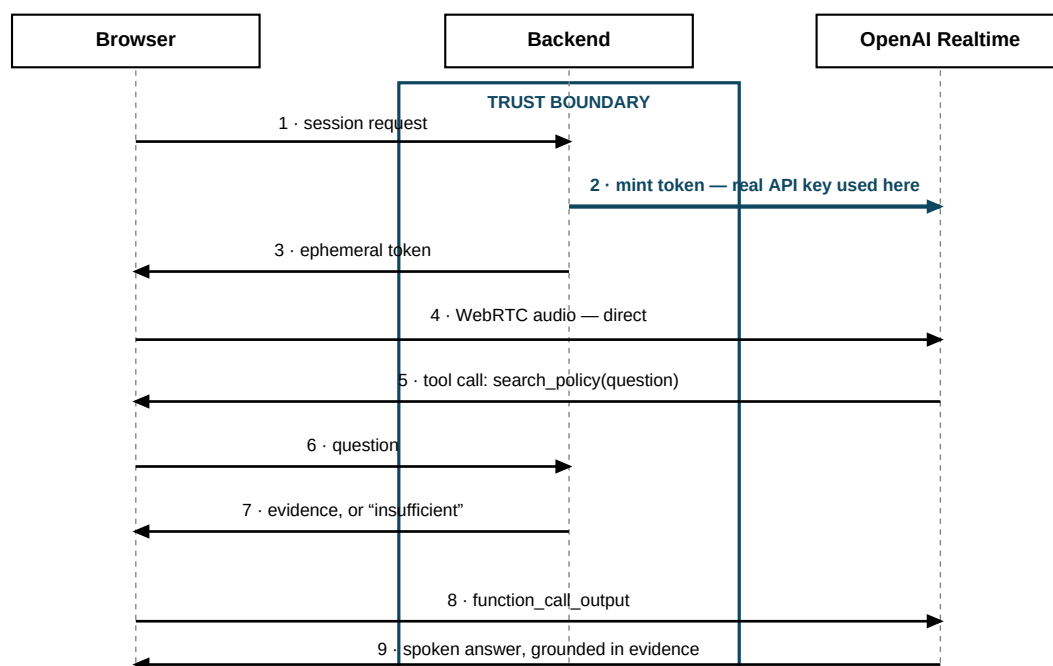
The information contained in the document is confidential, privileged and only for the information of the intended recipient and may not be used, published or redistributed without the prior written consent of LUMIQ. This proposal is the property of LUMIQ. No part of this document, including all concepts and ideas contained herein, may be reproduced or transmitted in any form, or by any means, electronic or mechanical, for any purpose without the express written permission of LUMIQ. Any non-consensual use, copying, republication or redistribution of contents contained in this document is expressly prohibited and are protected under applicable law.

All rights reserved.

Goal

A browser app where one speech-to-speech model explains a single health-insurance policy document out loud, lets the user interrupt at any point with a question, and answers every policy question strictly from that document — never from general knowledge, and never a guess.

Functional Architecture



The real API key (blue boundary) never leaves the backend. Audio runs straight between browser and model for low latency (step 4); every policy question is forced through the tool round-trip (steps 5–9) so the model can only answer from retrieved evidence.

Design Principles

- **One realtime model, no pipeline stitching.** Speech-to-text, reasoning, text-to-speech, and interruption handling are one model over one WebRTC connection — not three services glued together.
- **Evidence before answer.** The system prompt forces a search_policy tool call for any policy-specific question; the model is never allowed to answer from general insurance knowledge.

- **A fixed refusal, not a guess.** When retrieval confidence falls below threshold, the model returns one exact sentence — never an improvised, possibly wrong, answer.
- **Credentials and data stay server-side.** The browser only ever holds a short-lived session token and plain evidence text — never the API key, never the vector index.
- **The retriever is swappable.** One interface, `PolicyRetriever.search(question)`, isolates today's local JSON store from tomorrow's hosted vector search — nothing else in the system needs to change.

Solution Approach

Concern	Choice	Why
Voice model	OpenAI Realtime API (gpt-realtime) over WebRTC	Single model covers STT, reasoning, TTS, and barge-in — no LiveKit, no Pipecat, no multi-agent orchestration. Preferred over Gemini Live API because it connects browser-to-model over native WebRTC, versus Gemini's WebSocket transport, which needs a hand-built raw-PCM audio pipeline in the browser.
Auth	Backend mints a short-lived ephemeral token per session	The real API key is used exactly once per session, server-side, and never reaches the browser.
Tool execution	Browser relays the tool call; backend does the retrieval	This is how Realtime function-calling works — but the vector index and embeddings never leave the backend.
RAG store	Local JSON file, brute-force cosine similarity	Simplest reliable option for one policy document at POC scale — no database server, no ops.
Ingestion	One-off script: PDF -> chunks -> embeddings -> index	Runs once at setup; keeps the runtime path free of PDF parsing.

Security & Guardrails

- Browser holds only an ephemeral Realtime token — never the actual API key.
- Every policy question is forced through `search_policy`; the model cannot see or query the vector store directly.
- Answers are constrained to retrieved evidence by the system prompt, with one fixed fallback sentence when evidence is insufficient.